

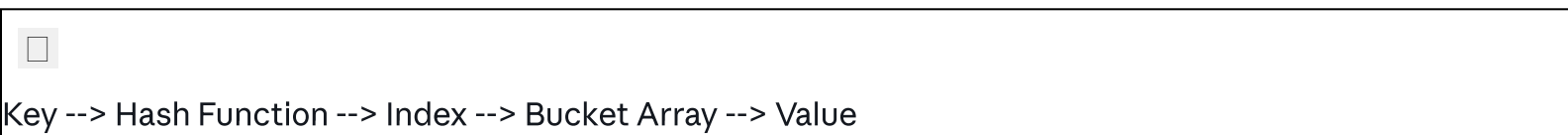
Data Structures Notes: Hashing & Trees

1. Hash Tables

Definition

A **Hash Table** (or Hash Map) is a data structure that stores data in an associative manner, using **key-value pairs**. It uses a **hash function** to compute an index (called a "hash code" or "bucket index") into an array of buckets, from which the desired value can be found. This allows for near constant-time complexity, $O(1)$ on average, for insertion, deletion, and lookup operations.

Structure



Types of Hash Tables

Open Hashing (Separate Chaining based) – Each bucket stores a list/chain of entries.

Closed Hashing (Open Addressing based) – All entries are stored within the array itself; on collision, the algorithm searches for the next open slot.

Perfect Hash Table – A hash table with no collisions at all (used when the key set is static and known in advance).

Dynamic Hash Table – Resizes itself (rehashing) as the number of elements grows, to maintain performance.

Example

Storing student roll numbers and names:

Key (Roll No.)	Hash Function (key % 7)	Index	Value
10	$10 \% 7$	3	"Amit"
25	$25 \% 7$	4	"Riya"
31	$31 \% 7$	3	"Sana" (collision at index 3)

Time Complexity

Operation Average Worst Case

Insert	$O(1)$	$O(n)$
--------	--------	--------

Operation Average Worst Case

Search $O(1)$ $O(n)$

Delete $O(1)$ $O(n)$

2. Hash Functions

Definition

A **Hash Function** is a function that takes an input (or "key") and returns a fixed-size integer value called a **hash code**, which is then mapped to an index in the hash table's array. A good hash function distributes keys uniformly across the table to minimize collisions.

Properties of a Good Hash Function

Deterministic – Same key always produces the same hash value.

Uniform distribution – Spreads keys evenly across buckets.

Efficient to compute – Should run in $O(1)$ time.

Minimizes collisions – Different keys should ideally map to different indices.

Uses the entire key – Should not ignore parts of the input.

Types of Hash Functions

Division Method $h(k) = k \bmod m$ (m = table size, usually a prime number) Example: $h(27) = 27 \bmod 10 = 7$

Multiplication Method $h(k) = \text{floor}(m * (k * A \bmod 1))$, where A is a constant ($0 < A < 1$). Used when table size is not a power of two.

Mid-Square Method Square the key, then extract the middle digits as the index. Example: key = 44 $\rightarrow 44^2 = 1936 \rightarrow$ middle digits "93" $\rightarrow$ index.

Folding Method Split the key into parts, add the parts, and use the result (mod table size) as the index. Example: key = 123456 $\rightarrow 123 + 456 = 579 \rightarrow h = 579 \bmod m$.

Cryptographic Hash Functions (e.g., MD5, SHA-256) Used for security purposes (password storage, checksums) rather than simple table indexing.

3. Collision Resolution Techniques

Definition

A **collision** occurs when two different keys are hashed to the same index in the hash table. Since two values cannot occupy the same slot in certain implementations, **collision resolution techniques** are used to handle this situation.

Types

A. Separate Chaining (Open Hashing)

Each bucket points to a linked list (or another structure) of all elements that hash to that index.


Index 3 --> [10 "Amit"] --> [31 "Sana"] --> NULL

Pros: Simple, table never "fills up."

Cons: Extra memory for pointers; worst case degrades to $O(n)$ if many collide.

B. Open Addressing (Closed Hashing)

All elements are stored inside the table itself. On collision, the algorithm probes for the next free slot.

Linear Probing $h(k, i) = (h(k) + i) \bmod m$ Check next slot sequentially ($i = 0, 1, 2, \dots$).

Issue: **Primary clustering** (long runs of occupied slots).

Quadratic Probing $h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$ Checks slots at increasing quadratic distances.

Reduces primary clustering but can cause **secondary clustering**.

Double Hashing $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$ Uses a second hash function to determine the probing step size.

Best distribution among open addressing methods.

C. Other Techniques

Rehashing - When the load factor (n/m) exceeds a threshold, a new larger table is created, and all elements are re-inserted using a new hash function.

Robin Hood Hashing - A variant of open addressing where entries can be swapped to minimize the variance of probe distances.

Example (Linear Probing)

Table size = 5, Keys to insert: 12, 17, 22

$h(12) = 12 \% 5 = 2 \rightarrow$ index 2

$h(17) = 17 \% 5 = 2 \rightarrow$ collision $\rightarrow$ check index 3 (empty) $\rightarrow$ place at 3

$h(22) = 22 \% 5 = 2 \rightarrow$ collision $\rightarrow$ index 3 occupied $\rightarrow$ check index 4 $\rightarrow$ place at 4

4. HashMaps

Definition

A **HashMap** is a concrete implementation of a hash table found in most programming languages (e.g., `HashMap` in Java, `dict` in Python, `unordered_map` in C++). It stores data as **key-value pairs**, where each key is unique, and provides efficient lookup, insertion, and deletion.

Key Characteristics

Keys are **unique**; values can be duplicated.

Unordered — does not maintain insertion order (unless a variant like `LinkedHashMap` is used).

Allows one `null` key and multiple `null` values in some languages (e.g., Java's `HashMap`).

Not thread-safe by default (`ConcurrentHashMap` is the thread-safe alternative in Java).

Types of Map Implementations

HashMap – Unordered, allows null keys/values, fast $O(1)$ average access.

LinkedHashMap – Maintains insertion order using a doubly linked list internally.

TreeMap – Maintains keys in sorted order (implemented via a Red-Black Tree), $O(\log n)$ operations.

ConcurrentHashMap – Thread-safe version for multi-threaded environments.

Example (Python dict)

```
python
student_marks = {
    "Amit": 85,
    "Riya": 92,
    "Sana": 78
}

print(student_marks["Riya"]) # Output: 92
student_marks["Kabir"] = 88 # Insert new key-value pair
del student_marks["Sana"]   # Delete a key
```

Example (Java HashMap)

```
java
HashMap<String, Integer> map = new HashMap<>();
map.put("Amit", 85);
map.put("Riya", 92);
System.out.println(map.get("Riya")); // Output: 92
```

5. Binary Trees

Definition

A **Binary Tree** is a hierarchical, non-linear data structure in which each node has **at most two children**, referred to as the **left child** and **right child**.

Terminology

Root – Topmost node.

Parent/Child – Relationship between connected nodes.

Leaf Node – A node with no children.

Height – Number of edges on the longest path from a node to a leaf.

Depth – Number of edges from the root to a node.

Types of Binary Trees

Full Binary Tree – Every node has either 0 or 2 children (never 1).

Complete Binary Tree – All levels are completely filled except possibly the last, which is filled left to right.

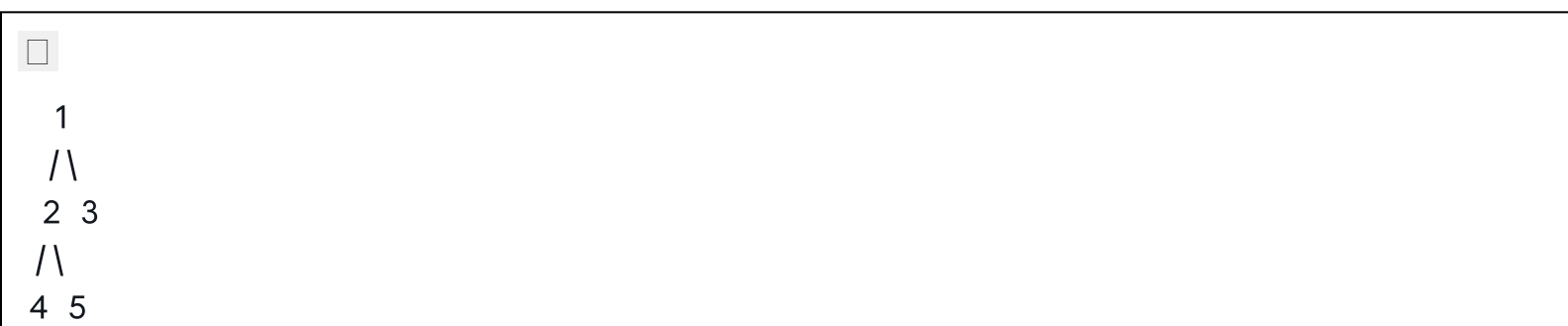
Perfect Binary Tree – All internal nodes have 2 children, and all leaves are at the same level.

Balanced Binary Tree – Height of the left and right subtrees of any node differs by at most 1 (e.g., AVL Tree).

Degenerate (Pathological) Tree – Each parent node has only one child, resembling a linked list.

Skewed Binary Tree – A degenerate tree skewed entirely to the left or right.

Example



This is a **Complete Binary Tree** since all levels are filled except the last, which fills left to right.

Basic Node Structure (C++)

```
cpp
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

6. Binary Search Trees (BST)

Definition

A **Binary Search Tree (BST)** is a special type of binary tree where, for every node:

All values in the **left subtree** are **less than** the node's value.

All values in the **right subtree** are **greater than** the node's value.

Both left and right subtrees are also BSTs.

This ordering property allows for efficient searching, insertion, and deletion — $O(\log n)$ on average.

Types / Variants of BST

Simple BST – The basic form described above; can become unbalanced (skewed).

Self-Balancing BSTs

AVL Tree – Balances itself using rotations based on the "balance factor."

Red-Black Tree – Uses node coloring rules to maintain balance (used in TreeMap, C++ `map`).

Splay Tree – Recently accessed nodes are moved to the root.

Threaded BST – Null pointers are replaced with pointers to the inorder predecessor/successor for faster traversal.

Example

Inserting 50, 30, 70, 20, 40, 60, 80:



Operations & Complexity

Operation Average Worst Case (Skewed)

Search $O(\log n)$ $O(n)$

Insert $O(\log n)$ $O(n)$

Delete $O(\log n)$ $O(n)$

Example Code (Insertion in C++)

```
cpp
```

```

Node* insert(Node* root, int key) {
    if (root == NULL) {
        Node* temp = new Node();
        temp->data = key;
        temp->left = temp->right = NULL;
        return temp;
    }
    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);
    return root;
}

```

7. Tree Traversals (Inorder, Preorder, Postorder)

Definition

Tree traversal refers to the process of visiting (checking/updating) each node in a tree data structure exactly once, in a systematic order. These are typically implemented using **Depth-First Search (DFS)** logic.

Types

A. Inorder Traversal (Left → Root → Right)

Visits the left subtree, then the root, then the right subtree.

For a BST, inorder traversal produces nodes in **sorted (ascending) order**.

☐

```

Inorder(node):
    if node is null: return
    Inorder(node.left)
    visit(node)
    Inorder(node.right)

```

B. Preorder Traversal (Root → Left → Right)

Visits the root first, then left subtree, then right subtree.

Used to create a **copy of the tree** or to get a **prefix expression** of an expression tree.

☐

```
Preorder(node):
    if node is null: return
    visit(node)
    Preorder(node.left)
    Preorder(node.right)
```

C. Postorder Traversal (Left → Right → Root)

Visits left subtree, then right subtree, then the root.

Used to **delete a tree** (children removed before parent) or get a **postfix expression**.

```
□
Postorder(node):
    if node is null: return
    Postorder(node.left)
    Postorder(node.right)
    visit(node)
```

D. Level Order Traversal (Breadth-First)

Visits nodes level by level, from top to bottom, left to right, using a queue.

Example

Given the tree:

```
□
  1
 / \
2   3
/ \
4  5
```

Inorder: 4, 2, 5, 1, 3

Preorder: 1, 2, 4, 5, 3

Postorder: 4, 5, 2, 3, 1

Level Order: 1, 2, 3, 4, 5

Time Complexity

All traversals run in **O(n)** time, since every node is visited exactly once. Space complexity is O(h) for recursive traversal (h = height of tree, due to the call stack).

8. Heaps

Definition

A **Heap** is a special, **complete binary tree**-based data structure that satisfies the **heap property**. It is commonly implemented using an array (not linked nodes), which makes it memory-efficient.

Heap Property

Max-Heap: The value of each parent node is **greater than or equal to** the value of its children. The maximum element is always at the root.

Min-Heap: The value of each parent node is **less than or equal to** the value of its children. The minimum element is always at the root.

Types of Heaps

Max-Heap – Root holds the maximum value.

Min-Heap – Root holds the minimum value.

Binary Heap – A heap where each node has at most 2 children (most common form).

Binomial Heap – A collection of binomial trees; supports efficient "merge" operations.

Fibonacci Heap – Offers better amortized time complexity for `decrease-key` and `merge`; used in advanced graph algorithms like Dijkstra's and Prim's.

Example (Min-Heap)

Array: [10, 15, 30, 40, 50, 100]

Tree Representation:

10

/ \

15 30

/ \ /

40 50 100

Array indexing: for node at index i , left child = $2i+1$, right child = $2i+2$, parent = $(i-1)/2$.

Key Operations

Operation	Description	Time Complexity
Insert	Add element at end, then "heapify up"	$O(\log n)$
Extract Min/Max	Remove root, move last element to root, "heapify down"	$O(\log n)$
Peek	View root element without removing	$O(1)$
Build Heap	Convert an arbitrary array into a heap	$O(n)$

Real-World Applications of Heaps

Implementing **Priority Queues**.

Heap Sort algorithm ($O(n \log n)$).

Finding the **k-th largest/smallest** element efficiently.

Graph algorithms: **Dijkstra's Shortest Path**, **Prim's Minimum Spanning Tree**.

9. Priority Queues

Definition

A **Priority Queue** is an abstract data type similar to a regular queue, but where each element has an associated **priority**. Elements with **higher priority** are served (dequeued) before elements with lower priority, regardless of their order of insertion.

Types

Max Priority Queue – The element with the highest priority (largest value) is served first.

Min Priority Queue – The element with the lowest priority (smallest value) is served first.

Based on implementation:

Array-based – Simple, but $O(n)$ for insertion/deletion to maintain order.

Linked List-based – Similar limitations to array-based.

Heap-based – Most common and efficient; $O(\log n)$ insertion/deletion.

Binary Search Tree-based – Also possible, $O(\log n)$ average.

How It's Usually Implemented

Priority Queues are almost always implemented internally using a **Heap** because a heap provides the fastest way to always access and remove the highest/lowest priority element.

Example

Hospital emergency room queue (Max Priority Queue by severity):

Patient Severity (Priority)

A 3

B 9 (Critical)

C 5

Even though A arrived first, **B is treated first** because of the highest priority (9).

Example Code (Python using `heapq`)



python

```
import heapq

pq = []
heapq.heappush(pq, (3, "Patient A"))
heapq.heappush(pq, (9, "Patient B"))
heapq.heappush(pq, (5, "Patient C"))

# heapq is a min-heap by default, so use negative priority for max-heap behavior
print(heapq.heappop(pq)) # Output: (3, 'Patient A') -- lowest number popped first
```

Applications of Priority Queues

CPU Scheduling in operating systems.

Dijkstra's Algorithm (shortest path) and **Prim's Algorithm** (MST).

Huffman Encoding for data compression.

Bandwidth management in networking (prioritizing certain data packets).

Simulation systems (event-driven simulations processed by time/priority).

10. Applications of Trees

Trees are used extensively in computer science because of their hierarchical, efficient-search nature.

Below are major real-world and computational applications:

1. File Systems

Operating systems represent directories and files as a tree structure (folders containing subfolders and files).

2. Databases

B-Trees and **B+ Trees** are used in database indexing (e.g., in MySQL, PostgreSQL) to allow fast search, insertion, and range queries on disk-based storage.

3. Binary Search Trees / Self-Balancing Trees

Used for fast searching, sorting, and maintaining ordered data — e.g., Java's `TreeMap`/`TreeSet` (Red-Black Trees).

4. Heaps and Priority Queues

Used in scheduling algorithms, load balancing, and pathfinding (Dijkstra's, Prim's).

5. Trie (Prefix Tree)

A tree used for efficient storage and retrieval of strings — used in **autocomplete**, **spell checkers**, and **IP routing (longest prefix matching)**.

6. Expression Trees

Used by compilers to parse and evaluate mathematical or logical expressions (e.g., $(a + b) * c$).

7. Huffman Coding Trees

Used in **data compression** algorithms (e.g., ZIP, JPEG) to assign shorter codes to more frequent characters.

8. Decision Trees

Used extensively in **Machine Learning** for classification and regression tasks.

9. Syntax Trees (Abstract Syntax Trees - AST)

Used by compilers and interpreters to represent the grammatical structure of source code.

10. Network Routing

Spanning Trees (e.g., via Prim's or Kruskal's algorithm) are used in network design to connect all nodes with minimum cost and no cycles.

11. Game Development / AI

Game Trees (like Minimax trees) are used in AI decision-making, such as in Chess or Tic-Tac-Toe engines.

Quick Summary Table

Topic	Core Idea	Time Complexity (avg)	Common Use Case
Hash Table	Key-value storage via hash function	$O(1)$	Fast lookups
Hash Function	Maps key to index	$O(1)$	Indexing
Collision Resolution	Handles hash clashes	$O(1)$ – $O(n)$	Chaining, probing
HashMap	Language implementation of hash table	$O(1)$	Dictionaries, caching
Binary Tree	Nodes with ≤ 2 children	$O(n)$ traversal	Hierarchical data
BST	Ordered binary tree	$O(\log n)$	Searching, sorting
Tree Traversal	Visiting order of nodes	$O(n)$	Serialization, evaluation
Heap	Complete tree with heap property	$O(\log n)$	Priority queues, sorting
Priority Queue	Serves by priority, not order	$O(\log n)$	Scheduling, pathfinding

Topic	Core Idea	Time Complexity (avg)	Common Use Case
Tree Applications	Real-world hierarchical use cases	Varies	File systems, DBs, compilers